

Master Architecture, Build Guide & Adversarial Defence Manual

ENGINEERING CREED

CORE PROJECT DIRECTIVE & PHILOSOPHY:

"Build one strong flow. Start small and deterministic. Ground every output in received context.
Make Vera hard to break before making Vera fancy. Never hardcode the visible examples.
Assume the real judge will try to break the system with unseen inputs."

Document Version:	2.2 (Post Phase 2B.2 Hardening & Gate)
Target System:	magicpin Vera AI Challenge Backend Service
Status:	PHASE 2B.2 GATE COMPLETE (34/34 AUTOMATED TESTS PASSING)
Core Framework:	Deterministic 4-Context Synthesis Engine + SQLite State Layer (FastAPI)
Source Repository:	c:/projects/magicpin (Production logic in app/)
Generated:	August 2026

TABLE OF CONTENTS	
1. Executive Overview & 4-Context Model	11. Adversarial Resilience & Break Matrix
2. Official Challenge Requirements & Contracts	12. Test Audit & Verification Architecture
3. Non-Negotiable Vera Operating Rules (30 Rules)	13. Real Judge Simulation Lifecycle
4. Technology Stack & Design Boundaries	14. Status Matrix: Complete vs Future
5. Actual Repository File Tree & Component Audit	15. Future LLM Integration Boundaries
6. End-to-End System Architecture	16. Future Multi-Turn Conversation Engine
7. Context Ingestion & Versioning Protocol	17. Customer-Facing Engagement Flow
8. Deterministic research_digest Flowchart	18. Project Roadmap (Phases 1 to 6)
9. Separation of Decision & Message Strategy	19. 'Where We Are Right Now' Summary
10. Multi-Merchant Suppression Architecture	20. Master Principle & Vera Glossary

1. Executive Overview & 4-Context Model

Vera is magicpin's autonomous engagement intelligence. Unlike generic conversational chatbots that hallucinate plausible answers from prompt memory, Vera is a **deterministic, context-grounded message engine**. Vera determines the exact next message, actionable Call-to-Action (CTA), send-as identity, suppression key, and clinical/business rationale for local merchants and customers.

CONTEXT SCOPE	CONTENTS & ROLE IN COMPOSITION	PRIMARY USE
Category Context (Macro Vertical)	Industry vertical voice profile, tone (peer_clinical / warm_practical), register, allowed & taboo vocabulary, dynamic salutation templates, curated research digest items, and patient/customer content library.	Sets communication register and supplies authoritative facts.
Merchant Context (Business Account)	Business name, owner first name, city, locality, verification status, languages, active subscription tier, CTR/views performance metrics, customer aggregates, active offers, cohort signals, and conversation history.	Anchors relevance to merchant identity and audience signals.
Trigger Context (Event Signal)	Event kind (research_digest, churn_risk, festival, new_offer, view_spike), urgency (1-5), target item IDs, category slug, expiration timestamp, and vertical/merchant suppression key.	Dictates timing, trigger reason, and suppression identity.
Customer Context (Optional Consumer)	Customer identity, preferences, visit cadence, spend aggregate, lifetime value, and consent records.	Enables personalized customer outreach in Phase 4.

2. Official Challenge Requirements & Contracts

DIMENSION	OFFICIAL CHALLENGE SPECIFICATION
Hard Requirements	Implement standard HTTP REST endpoints on FastAPI. Enforce strict ACID SQLite context storage with version conflict detection (409 on stale version, 200 on duplicate idempotent). Enforce multi-tenant suppression deduplication. Respect trigger expiration.
Scoring Dimensions (50 pts)	1. Specificity (10): Verifiable numbers, trial sizes, citations. 2. Category Fit (10): True vertical voice, zero taboo words. 3. Merchant Fit (10): Owner name, cohort signals, language. 4. Decision Quality (10): Correct trigger routing, suppression. 5. Engagement (10): Compelling, low-friction, single CTA.
Required Endpoints	GET /v1/healthz — Diagnostic status and exact counts of loaded contexts. GET /v1/metadata — Team name, architecture summary, operational model. POST /v1/context — Ingestion of category, merchant, customer, trigger contexts. POST /v1/tick — Periodic wake-up evaluation emitting up to 20 actions. POST /v1/reply — Multi-turn conversation reply handler.
Operational Caps	Max 20 actions per tick. Max timeout: 5s for healthz/metadata, 10s for context, 15s for tick/reply. Latencies must remain <100ms.

3. Non-Negotiable Vera Operating Rules (30 Rules)

1. Ground everything strictly in received context.	16. Never repeat messages due to state bugs.
2. Never invent numbers, sample sizes, or stats.	17. Respect merchant-scoped suppression keys.
3. Never invent offers, discounts, or vouchers.	18. Respect merchant opt-outs and unsubscribe history.
4. Never invent dates, deadlines, or schedules.	19. YES / GO AHEAD leads directly to execution.
5. Never invent customer facts or preferences.	20. Repeated auto-replies must back off and end.
6. Never invent business metrics or claim counts.	21. Customer outreach requires explicit consent context.
7. Never invent clinical claims or medical advice.	22. Higher context versions atomically replace stale ones.
8. Handle Category + Merchant + Trigger correctly.	23. Same input + same state = identical deterministic behavior.
9. Decide before writing — logic dictates copy.	24. Keep API responses sub-10ms (fast & lightweight).
10. Specificity > generic marketing fluff.	25. Never hardcode visible sample strings in logic.
11. Merchant fit > superficial personalization.	26. Never test-fit to pass the local judge simulator.
12. One strong, low-friction CTA per message.	27. Real judge injects unseen merchants and verticals.
13. Sometimes WAIT is the optimal answer.	28. Passing unit tests does not equal judge immunity.
14. Sometimes SUPPRESS is the optimal answer.	29. Build small & deterministic before adding complexity.
15. Sometimes END is the optimal answer.	30. Make Vera hard to break before making Vera fancy.

EXECUTION PIPELINE

THE CORE VERA DECISION PIPELINE:

GROUND (Verify 4 Contexts) → **DECIDE** (Gating & Eligibility) → **SELECT FACTS** (Digest & Signals) → **COMPOSE** (Rule-Based Synthesis) → **VALIDATE** (Word-Boundary Taboo Filter) → **RESPOND SAFELY** (Record Suppression & Emit)

4. Technology Stack & Design Boundaries

LAYER / COMPONENT	TECHNOLOGY USED	RATIONALE & CONSTRAINTS
Runtime & API	Python 3.11 + FastAPI + Uvicorn	Asynchronous, high-performance, strictly typed REST endpoints with automatic OpenAPI schema generation.
Data Validation	Pydantic v2	Contract validation enforcing non-nullable scopes, ISO dates, integer versions, and strict action models.
Persistent Storage	SQLite (WAL Mode + Thread Lock)	Zero-dependency ACID persistence, composite primary key deduplication, atomic version transitions, and sub-millisecond lookups.
Testing Framework	pytest + FastAPI TestClient	34 automated tests spanning unit contracts, flow gating, adversarial edge cases, and judge simulator adapters.
Containerization	Docker (Multi-stage build)	Production-ready, portable image running on configured PORT with clean SQLite volume binding.
Intentionally Excluded	Vector DBs, Redis, LangChain, Multi-Agent Bloat	Unnecessary operational complexity. Deterministic routing and factual extraction do not require vector search.

5. Actual Repository File Tree & Component Audit

FILE PATH	PURPOSE & SCOPE	STATUS
app/main.py	FastAPI app entrypoint, lifespan startup (DB init), global error handlers, route inclusion.	COMPLETE
app/config.py	App constants, server port configuration, and Vera operating principles.	COMPLETE
app/models/context.py	Pydantic models for /v1/context push and response payloads.	COMPLETE
app/models/health.py	Pydantic models for /v1/healthz and /v1/metadata.	COMPLETE
app/models/interaction.py	Pydantic models for /v1/tick (TickAction) and /v1/reply.	COMPLETE
app/store/context_store.py	SQLite engine: atomic versioning, composite suppression tracking, legacy schema migration.	COMPLETE
app/engine/salutation.py	Dynamic greeting resolver driven by CategoryContext.voice and merchant identity.	COMPLETE
app/engine/composer.py	Deterministic composer for research_digest: factual extraction, cohort linking, taboo filters.	COMPLETE
app/routes/health.py	Route handlers for health status, context counts, and metadata inspection.	COMPLETE
app/routes/context.py	Route handler for POST /v1/context with 409 stale version rejection.	COMPLETE
app/routes/interaction.py	Route handler for /v1/tick (active composer) and /v1/reply (Phase 1/2 stub).	PARTIAL
app/engine/conversation.py	Multi-turn conversation engine and intent state machine for replies.	FUTURE

6. End-to-End System Architecture

COMPONENT	TYPE	INPUTS & OUTPUTS	STATUS
HTTP Gateway (FastAPI / Uvicorn)	REST Ingress	Receives Judge HTTP calls on /v1/context, /v1/tick, /v1/reply, /v1/healthz, /v1/metadata.	ACTIVE
Context Store (SQLite Engine)	State Layer	Stores scopes (category, merchant, customer, trigger). Manages version conflicts and suppression pairs.	ACTIVE
Decision & Gating (Rules Engine)	Deterministic Logic	Evaluates trigger expiry, vertical consistency, merchant subscription, opt-out history, and suppression.	ACTIVE
Factual Composer (Language Synthesizer)	Fact Synthesizer	Synthesizes lead hook, cohort relevance, trial facts (N=...), and topic-aware CTA without hallucinations.	ACTIVE
Safety Validator (Word Boundary Filter)	Taboo Scrubber	Filters category-specific taboo terms using \b word boundaries without corrupting legitimate words.	ACTIVE
Conversation Engine (Multi-Turn Reply)	State Machine	Handles merchant replies (YES/GO AHEAD, questions, auto-replies, hostile opt-outs).	PHASE 3

7. Context Ingestion & Versioning Protocol

Context arrives via POST /v1/context. The system stores the latest valid state per (scope, context_id) and rejects stale updates with HTTP 409 Conflict.

VERSION SCENARIO	STORE STATE	INCOMING VERSION	HTTP STATUS & ACTION
Initial Ingestion	Empty (no record)	Version 1	200 OK — Stored atomically into SQLite contexts table.
Idempotent Resend	Stored Version 1	Version 1	200 OK — Idempotent no-op. Returns accepted=true.
Context Update	Stored Version 1	Version 2 (or higher)	200 OK — Atomically overwrites payload and updates version.
Stale / Out-of-Order	Stored Version 2	Version 1 (lower)	409 Conflict — Rejected. Body contains error='stale_version'.
Invalid Scope	Any	scope='unknown'	400 Bad Request — Rejected. Must be category/merchant/customer/trigger.

8. Deterministic research_digest Flowchart

STEP / GATING STAGE	DETERMINISTIC EVALUATION LOGIC	FAILURE OUTCOME
1. Trigger Validation	Verify trigger scope == 'merchant' and kind == 'research_digest'.	SUPPRESS (No-op)
2. Expiry Check	Compare ISO timestamp body.now > trigger.expires_at.	SUPPRESS (Stale Trigger)
3. Entity Resolution	Fetch MerchantContext by merchant_id and CategoryContext by category_slug.	SUPPRESS (Missing Context)
4. Category Match	Verify trigger.payload.category == merchant.category_slug.	SUPPRESS (Vertical Mismatch)
5. Subscription & Opt-Out	Check active plan and inspect conversation_history for unsubscribe / stop signals.	SUPPRESS (Opted Out / Inactive)
6. Multi-Tenant Suppression	Query SQLite for (suppression_key, merchant_id).	SUPPRESS (Already Sent)
7. Fact & Cohort Linking	Match top_item_id in category.digest; link patient_segment to merchant signals.	SAFE FALLBACK (General Practice)
8. Salutation & Hook	Resolve greeting from category voice examples; extract journal/issue from source.	SAFE FALLBACK ('Doc' / 'Hi team')
9. Fact Synthesis	Format core finding with sample size (N=...) without double verbs.	SUPPRESS (Missing summary/title)
10. Topic-Aware CTA	Route CTA based on digest kind (compliance, tech, cde, trend, clinical).	Standard Low-Friction Ask
11. Word-Boundary Taboo	Clean taboo list; scrub standalone taboo terms with lb word boundaries.	Preserves legitimate words
12. Action Emission	Record (suppression_key, merchant_id); return structured TickAction in actions[].	Max 20 actions per tick

9. Separation of Decision & Message Strategy

A foundational design principle of Vera is the strict separation between **Deterministic Decision Making** and **Language Synthesis**. The decision engine evaluates eligibility and selects factual anchors; the composer transforms structured facts into natural prose. This guarantees zero hallucinations.

```
{ "merchant_id": "m_opt_kavita_pune", "salutation": "Dr. Kavita", "source_lead": "Optometry Vision Science's Nov issue landed.", "cohort_anchor": "One item relevant to pediatric myopia patients – ", "finding_fact": "Randomized 2-year trial demonstrated 59% reduction in axial elongation (N=1,560).", "cta_text": "Worth a look (2-min abstract). Want me to pull the key takeaways for your team?", "citation_footer": " – Optometry Vision Science Nov 2026, p.50", "suppression_key": "research:optometry:2026-w45", "rationale": "External research digest with pediatric cohort anchor. Source citation at end maintains clinical credibility." }
```

10. Multi-Merchant Suppression Architecture

In weekly digests and external announcements, triggers carry vertical-level suppression keys (e.g. research:dentists:2026-W17). If suppression were globally keyed on `suppression_key` alone, the first merchant would receive the message and all other merchants in that vertical would be silently dropped. Vera enforces **Merchant-Scoped Suppression**.

GLOBAL KEYING (VULNERABLE)	MERCHANT-SCOPED KEYING (VERA ARCHITECTURE)
PRIMARY KEY (<code>suppression_key</code>) • Merchant A evaluates key X → SENT • Key X is recorded globally. • Merchant B evaluates key X → BLOCKED Outcome: 49/50 merchants silently dropped.	PRIMARY KEY (<code>suppression_key</code>, <code>merchant_id</code>) • Merchant A evaluates key X → SENT (X, A) • Merchant B evaluates key X → SENT (X, B) • Merchant A receives tick 2 → SUPPRESSED Outcome: Perfect multi-tenant isolation.

11. Adversarial Resilience & Break Matrix

FAILURE VECTOR	JUDGE INJECTION	VERA DETERMINISTIC DEFENCE	STATUS
Expired Trigger	<code>now=2026-05-04, expires=2026-05-03</code>	Timestamp comparison detects staleness; yields no-op.	PASS
Category Mismatch	Salon trigger to dentist merchant	Category consistency check rejects action.	PASS
Opt-Out History	Merchant previously sent 'stop'	Conversation history scanner flags opt-out; suppresses.	PASS
Missing Owner Name	<code>owner_first_name=null</code> (doctor)	Resolves to category fallback 'Doc' (never 'Dr. None').	PASS
Missing Trial Size	<code>trial_n=null</code> in digest item	Grounds in summary text without fabricating trial size.	PASS
Missing Summary	<code>trial_n=2100</code> but <code>summary=""</code>	Suppresses action; never fabricates clinical claims.	PASS
Unseen Journals	<code>Source="The Lancet Nov 2026"</code>	Dynamic parser handles publication/issue without whitelist.	PASS
Taboo Substrings	Text with 'secure', 'accurate'	Word-boundary regex (<code>\b</code>) preserves legitimate words.	PASS
Trigger Flood (21+)	30 candidate triggers in <code>/v1/tick</code>	Prioritizes by urgency descending; caps strictly at 20.	PASS
Duplicate Triggers	<code>available_triggers=['trg_1','trg_1']</code>	First trigger records suppression; second is deduplicated.	PASS
Stale Version Resend	Pushing v1 after v2 stored	Rejects with HTTP 409 Conflict (<code>stale_version</code> error).	PASS

12. Test Audit & Verification Architecture

TEST SUITE FILE	TEST TYPE & PURPOSE	COUNT	STATUS
tests/test_context.py	Contract tests for context scopes, atomic updates, version replacement, stale rejection.	7 tests	7/7 PASS
tests/test_health.py	Diagnostic tests for /v1/healthz counts, metadata team details, and root endpoint.	4 tests	4/4 PASS
tests/test_interaction.py	API contract stubs for /v1/tick and /v1/reply interaction lifecycle.	2 tests	2/2 PASS
tests/test_flow_research_digest.py	Adversarial flow tests: factual synthesis, word-boundary taboos, multi-vertical salutations.	12 tests	12/12 PASS
tests/test_judge_simulation_gate.py	Integration gate tests: full 355 context warmup, persistence, multi-merchant isolation.	8 tests	8/8 PASS
tests/test_judge_sim_runner.py	Judge simulator runner executing warmup and phase2_short scenarios against backend.	1 test	1/1 PASS
TOTAL VERIFICATION	Comprehensive regression & judge gate suite executing in ~12.2s.	34 tests	34/34 PASS

CRITICAL PRINCIPLE: 'Tests written by us are necessary, but they are not the same thing as the real judge. True robustness requires zero hardcoding and complete generalizability across unseen contexts.'

13. Real Judge Simulation Lifecycle

STAGE	JUDGE ACTIONS & PROTOCOL	KEY SCORING CRITERIA
1. Warmup	Calls GET /v1/healthz and GET /v1/metadata. Pushes initial batch of categories and merchants via POST /v1/context.	Sub-100ms latency, exact reflection of loaded context counts in healthz.
2. Ingestion	Pushes 50 merchants, 200 customers, 100 triggers. Injects updated versions (v2) and stale versions (v1).	Strict 200 on valid versions, 409 Conflict on stale versions.
3. Tick	Calls POST /v1/tick with simulated timestamp advances and batches of available trigger IDs.	Grounded specificity, correct salutation, taboo scrubbing, max 20 action cap.
4. Reply Flow	Sends merchant responses (positive YES, clarification questions, repeated auto-replies, hostile opt-outs).	Auto-reply backoff detection, opt-out suppression, actionable flow completion.
5. Scoring	LLM Judge evaluates 5 dimensions (0-10 each). Imposes heavy penalties for hallucinations, spam, or broken state.	50 points maximum. Target score: >80% (Excellent).

14. Status Matrix: Complete vs Future

COMPLETE & HARDENED (PHASE 1 - 2B.2)	SCHEDULED FOR FUTURE PHASES (PHASE 3 - 6)
• Typed Pydantic models for all 5 endpoints • ACID SQLite store with atomic versioning • 409 Conflict on stale context versions • Multi-tenant (suppression_key, merchant_id) store • Diagnostic /healthz & /metadata endpoints • Deterministic research_digest composition • Dynamic Category voice salutation engine • Generic journal hook parser (no whitelist) • Robust fact synthesis with (N=...) integration • Word-boundary taboo filter (lb) • Topic-aware CTA router across 5 kinds • Strict 20-action cap with urgency ranking • 34 automated unit, flow & gate tests passing	• Phase 3: Multi-turn /v1/reply conversation engine • Phase 3: YES / GO AHEAD execution state machine • Phase 3: Auto-reply backoff & loop termination • Phase 3: Merchant hostile opt-out state tracking • Phase 4: Churn risk trigger family (m_churn) • Phase 4: Festival & holiday triggers (trg_festival) • Phase 4: Offer drop & view spike triggers • Phase 4: Customer-facing outreach flows • Phase 5: Live LLM provider integration & validator • Phase 5: Replay testing against full challenge suite • Phase 6: Production Docker deployment & submission

15. Future LLM Integration Boundaries

When an LLM provider is introduced in Phase 5, it will operate inside strict deterministic boundaries. The LLM is **never trusted with state, suppression, or factual grounding**.

WHAT THE LLM IS NEVER TRUSTED TO DECIDE	WHAT THE LLM IS USED FOR
• Context existence or validity • Version conflict resolution (409 Conflict) • Suppression eligibility or duplicate checks • Category consistency or merchant verification • Selecting trial numbers, metrics, or offers • API contract adherence or action limits	• Natural phrasing and conversational transitions • Code-mixing (Hindi + English natural phrasing) • Polite clarification questions on ambiguous merchant replies • Dynamic rephrasing within strict MessageStrategy tokens • Empathy adjustments matching Category tone

SANDWICH PATTERN

FUTURE LLM SANDWICH ARCHITECTURE:

Raw Context → Deterministic Gating → MessageStrategy Struct → LLM Paraphrase → Deterministic Taboo & Fact Validator → Action Emission

16. Future Reply & Conversation Engine (Phase 3)

CONVERSATION STATE	TRIGGER / INPUT	TRANSITION & BOT ACTION
OUTBOUND_SENT	Proactive tick message emitted	State set to WAITING_REPLY; turn_number=1.
POSITIVE_AFFIRMATION	Merchant replies 'Yes', 'Sure', 'Send it'	Transition to ACTION_EXECUTION; deliver draft artifact; set status=COMPLETED.
CLARIFICATION_ASK	Merchant asks 'What is the dosage?'	Extract factual detail from digest; answer concisely; repeat low-friction CTA.
AUTO_REPLY_DETECTED	Simulated automated greeting received	Increment auto_reply_count; back off (no response or gentle 1-time acknowledgment); terminate after 2 turns.
HOSTILE_OPT_OUT	Merchant replies 'Stop messaging me'	Acknowledge respectfully; record suppression in DB; terminate conversation.

17. Customer-Facing Engagement Flow (Phase 4)

In Phase 4, Vera expands to consumer-facing engagement. While merchant messaging focuses on business enablement and clinical insights, customer messaging focuses on **personalized offers, loyalty rewards, re-engagement, and service reminders** grounded in CustomerContext preferences.

ATTRIBUTE	MERCHANT-FACING VERA	CUSTOMER-FACING VERA
Audience & Persona	Business owners, clinic directors, salon founders.	End consumers, patients, diners, gym members.
Tone & Voice	Peer-clinical, collegial, ROI-focused, respectful.	Warm, inviting, concise, value-oriented.
Primary Triggers	research_digest, churn_risk, ctr_drop, view_spike.	visit_lapse, birthday, festival_offer, voucher_drop.
CTA Goal	Approve draft campaign, audit protocol, update offer.	Book appointment, claim voucher, order now.

18. Project Roadmap (Phases 1 to 6)

PHASE	GOAL & KEY DELIVERABLES	STATUS
Phase 1: Foundation	FastAPI app scaffolding, Pydantic contracts, SQLite context store with 409 stale version rejection, healthz and metadata endpoints.	COMPLETE
Phase 2A: Analysis	Deep-dive specification analysis of research_digest trigger, failure modes, and 10 adversarial variations.	COMPLETE
Phase 2B: Initial Flow	First end-to-end research_digest flow connected to /v1/tick and SQLite suppression tracking.	COMPLETE
Phase 2B.1: Hardening	Eliminated all hardcoding, multi-tenant suppression, word-boundary taboos, generic journal hooks, dynamic category salutations, 20-action cap.	COMPLETE
Phase 2B.2: Integration Gate	Full 355 context warmup, simulator compatibility, persistence restart checks, latency profiling (34/34 tests passing).	COMPLETE
Phase 3: Reply Engine	Intelligent multi-turn /v1/reply handler, auto-reply backoff, YES action execution, hostile opt-out recording.	START
Phase 4: Remaining Triggers	Churn risk, view spike, festival offers, customer-facing outreach flows.	PLANNED
Phase 5: LLM Integration	Sandwich LLM paraphrase layer, replay validation against full test pairs.	PLANNED
Phase 6: Submission	Final Docker build, container validation, submission package delivery.	PLANNED

19. 'Where We Are Right Now' Executive Snapshot

QUESTION	EXACT CURRENT STATE (VERIFIED AGAINST REPOSITORY)
What Exists?	A fully operational, lightweight FastAPI service with ACID SQLite storage, typed Pydantic models, deterministic salutation and composition engines, and 34 automated unit and integration tests.
What Works?	Context ingestion (all 4 scopes), version conflict handling (409 on stale version), multi-merchant suppression deduplication, research_digest tick message synthesis, topic-aware CTAs, and sub-10ms response times.
What Are We Testing?	Full warmup with 355 contexts, adversarial edge cases (expired triggers, category mismatches, opt-out history, missing owner names, taboo boundary safety, action caps), and official judge simulator compatibility.
What Is Still A Stub?	POST /v1/reply currently returns a compliant stub (action='end'). Intelligent multi-turn conversational replies and auto-reply backoff are intentionally scheduled for Phase 3.
What Are We NOT Building Yet?	We are intentionally NOT adding vector databases, Redis, multi-agent frameworks, or ungrounded LLM prompts. We keep the core engine deterministic, robust, and fast first.
What Comes Next?	Phase 3: Multi-turn Reply Engine — implementing the conversation state machine, positive execution transitions, and auto-reply loop termination.
What Must NEVER Be Forgotten?	Ground every response strictly in received context. Never hardcode sample data. Make Vera hard to break before making Vera fancy.

20. Final Master Rule & Glossary of Vera Terms

TERM	DEFINITION IN THE VERA ENGINE
CategoryContext	Vertical-level context setting the communication voice, vocabulary rules, allowed/taboo terms, and research digest library.
MerchantContext	Account-level context capturing identity, owner name, locality, subscription status, performance metrics, and cohort signals.
TriggerContext	Event signal specifying trigger kind, urgency, expiration, and suppression key.
Suppression Key	Unique deduplication key scoped per merchant (suppression_key, merchant_id) to prevent repeat messages.
Gating	Deterministic pre-composition checks (expiry, category match, opt-out, subscription, suppression) returning safe no-ops on failure.
Factual Grounding	Strict rule that every number, statistic, journal name, and claim in the message body must originate from provided context.
Topic-Aware CTA	Call-to-action tailored to the digest item kind (compliance checklist, tech workflow, cde credits, or patient education WhatsApp).

FINAL ARCHITECTURAL LAW
THE VERA MASTER PRINCIPLE
*"Do not build Vera to answer the visible examples.
Build Vera so that the visible examples are merely natural consequences of correct, generalized rules."*